

Skill 6

1. To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.
To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_ARGS 100

// Parse command and handle escaped characters
int parse_command(char *input, char *args[])
{
    int argc = 0;
    char *src = input;
    char *dst = input;

    while (*src != '\0' && *src != '\n') {

        // Skip normal spaces
        if (*src == ' ') {
            src++;
            continue;
        }

        args[argc++] = dst;

        // Read one argument
        while (*src != '\0' && *src != '\n' && *src != ' ') {

            // Handle escape character
            if (*src == '\\') {
                src++;

                if (*src != '\0') {
                    *dst++ = *src++;
                }
            }
        }
    }
}
```

```

        } else {
            *dst++ = *src++;
        }
    }

    *dst++ = '\0';

    if (argc >= MAX_ARGS - 1)
        break;
}

args[argc] = NULL;

return argc;
}

int main()
{
    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    printf("Mini Shell Started\n");
    printf("Type 'exit' to quit.\n\n");

    while (1) {

        printf("myshell$ ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        // Remove newline
        input[strcspn(input, "\n")] = '\0';

        // Empty command
        if (strlen(input) == 0)
            continue;

        // Exit command
        if (strcmp(input, "exit") == 0)
            break;

        // Parse command

```

```

int argc = parse_command(input, args);

if (argc == 0)
    continue;

// Create child process
pid_t pid = fork();

if (pid < 0) {
    perror("fork failed");
    continue;
}

// Child process
if (pid == 0) {

    printf("[Child] PID = %d\n", getpid());

    // Execute command
    execvp(args[0], args);

    // If execvp fails
    perror("Execution failed");
    exit(EXIT_FAILURE);
}

// Parent process
else {

    int status;

    printf("[Parent] Created child PID = %d\n", pid);
    // Wait for child
    waitpid(pid, &status, 0);
    if (WIFEXITED(status)) {
        printf("[Parent] Child exited with status %d\n",
            WEXITSTATUS(status));
    }
    printf("\n");
}
}

printf("Shell terminated.\n");

```

```

return 0;
}

```

```

adithya@LAPTOP-07A78KDI: ~$ cd OS
adithya@LAPTOP-07A78KDI:~/OS$ parse_command()
> gcc skill6.c -o skill6
./skill6
-bash: syntax error near unexpected token `gcc'
-bash: ./skill6: No such file or directory
adithya@LAPTOP-07A78KDI:~/OS$ gcc skill6.c -o skill6
./skill6
cc1: fatal error: skill6.c: No such file or directory
compilation terminated.
-bash: ./skill6: No such file or directory
adithya@LAPTOP-07A78KDI:~/OS$ nano skill6.c
adithya@LAPTOP-07A78KDI:~/OS$ gcc skill6.c -o skill6
adithya@LAPTOP-07A78KDI:~/OS$ ls
5      Skill7      fifo_server      memory.c      practical4.c      server.c
Experiment Skill8      fifo_server.c    memory_layout practical5      shell
GIT      Skill9      file             memory_layout.c practical5.c      shell.c
OSSP     client     file.c           mini_shell    process_demo     signal_handler
Skill1    client.c    fork             mini_shell.c   process_demo.c   signal_handler.c
Skill10   client_to_server hello           os-lab         process_states   skill6
Skill11   command_history hello.c         output.txt      process_states.c skill6.c
Skill12   command_history.c keyboard_shell  parser          producer_consumer source.txt
Skill2    dynamic_memory keyboard_shell.c parser.c         producer_consumer.c tokenizer.c
Skill3    dynamic_memory.c keyboard_shell.c.save practical2      quotes          tokenzier.c
Skill4    experiment5  ls_grep         practical2.c   quotes.c         zombie
Skill5    fifo_client  ls_grep.c       practical3.c   sample.txt       zombie.c
Skill6    fifo_client.c memory          practical4     server

adithya@LAPTOP-07A78KDI:~/OS$ ./skill6
Mini Shell Started
Type 'exit' to quit.

mysshell$ exit
Shell terminated.
adithya@LAPTOP-07A78KDI:~/OS$ ./skill6
Mini Shell Started
Type 'exit' to quit.

mysshell$ ls
[Parent] Created child PID = 638
[Child] PID = 638
5      Skill7      fifo_server      memory.c      practical4.c      server.c
Experiment Skill8      fifo_server.c    memory_layout practical5      shell
GIT      Skill9      file             memory_layout.c practical5.c      shell.c
OSSP     client     file.c           mini_shell    process_demo     signal_handler

```

```

adithya@LAPTOP-07A78KD: .  ×  +  ▾

myshell$ ls
[Parent] Created child PID = 638
[Child] PID = 638
5      Skill7      fifo_server      memory.c      practical4.c      server.c
Experiment Skill8      fifo_server.c    memory_layout practical5      shell
GIT      Skill9      file      memory_layout.c practical5.c      shell.c
OSSP      client      file.c      mini_shell      process_demo      signal_handler
Skill11      client.c      fork      mini_shell.c      process_demo.c      signal_handler.c
Skill10      client_to_server hello      os-lab      process_states      skill6
Skill11      command_history hello.c      output.txt      process_states.c      skill6.c
Skill12      command_history.c keyboard_shell parser      producer_consumer      source.txt
Skill2      dynamic_memory keyboard_shell.c parser.c      producer_consumer.c      tokenizer
Skill3      dynamic_memory.c keyboard_shell.c.save practical2      quotes      tokenizer.c
Skill4      experiment5      ls_grep      practical2.c      quotes.c      zombie
Skill5      fifo_client      ls_grep.c      practical3.c      sample.txt      zombie.c
Skill6      fifo_client.c      memory      practical4      server
[Parent] Child exited with status 0

myshell$ pwd
[Parent] Created child PID = 639
[Child] PID = 639
/home/adithya/OS
[Parent] Child exited with status 0

myshell$ echo Hello\ World
[Parent] Created child PID = 640
[Child] PID = 640

[Parent] Child exited with status 0

myshell$ echo Hello\&World
[Parent] Created child PID = 641
[Child] PID = 641

[Parent] Child exited with status 0

myshell$ echo Hello\&World
[Parent] Created child PID = 642
[Child] PID = 642

[Parent] Child exited with status 0

myshell$ |

```